

KUBRA

Datafeed API

Last updated: July 31, 2025

© KUBRA — All Rights Reserved

Datafeed API

1.0 What do I need to upload files to the Datafeed API?

The Datafeed API allows you to upload your account and balance data to KUBRA. You need the following information before uploading your files to KUBRA Datafeed:

1. **Space KRN**: Provided by your KUBRA Service Delivery Team member or Project Manager.
2. **Machine client ID** and **client secret**: Provided by your KUBRA Project Manager or Service Delivery Team member.
3. **KUBRA client ID**: Provided by your KUBRA Project Manager or Service Delivery Team member.
4. **Machine token**: You can generate this token by taking the steps outlined in this document.
5. **Realm**: The virtual environment to differentiate between test and production files.
6. **Metatags (*Optional*)**: Key/value pairs to tag uploaded files for other KUBRA applications.

Within our production environment, KHQ uses two virtual environment types, also referred to as realms. Realms are of two types: **sandbox** and **live**. The sandbox realm is your testing environment, while the live realm is a production environment used by your consumers.

The URL **MUST** be `https`. Using an unencrypted `http` URL for the datafeed will result in `401` errors. Any credentials shared via an `http` URL will be assumed to be compromised.

2.0 How do I upload files to the Datafeed API?

After your KUBRA Implementation Team member has provided you with a space KRN, machine client ID, and client secret, you must retrieve a machine token (JWT token, bearer token, or access token) to make API calls to upload to the Datafeed API.

KUBRA APIs are HTTP-based **RESTful APIs** that use OAuth 2.0 for authentication and authorization to provide secure access to protected resources. API request and response bodies are formatted in JSON. Before you can use our APIs, you must obtain OAuth 2.0 client ID and secret credentials. These credentials are then exchanged for an access token that authorizes your REST API calls.

You cannot run the sample requests as they are. You will need to replace call-specific parameters such as tokens and IDs with your own values.

2.1 Obtaining client credentials

To make REST API calls, you will need to request your client credentials first. Client credentials include Client ID and Client Secret.

You can request them from your Project Manager. If you are not actively engaged in a project, you can create and submit a case through ServiceNow - Client Support. KUBRA will provide you with a specific set of client credentials.

KUBRA recommends the delivery of credentials using one of the following methods:

1. Phone
2. SMS
3. Password-protected file via email

Do not integrate KUBRA APIs directly in the front end of SPA applications as it will result in the disclosure of client credentials and/or access tokens.

2.2 Obtaining an access token

Once you have obtained your client credentials, you can request an access token by calling the KUBRA Authentication API, using such credentials. In response, an access token will be issued by KUBRA.

Use the URL below in a curl call or implement it within your code:

V2 Machines:

- https://api.kubra.io/authentication/api/v2/token

V1 Machines:

- https://api.kubra.io/authentication/api/v1/token

If you are having any issues or you're not sure which version your machine is (V1 or V2), reach out to your Project Manager or client support.

2.2.1 Access token request specs

Spec Category	Spec Detail	Comment
HTTP Verb	POST	This service does not require any query string parameters.
Params (query)	None	This service does not require any query parameters.
Params (path)	None	This service does not require any path parameters.
Authorization	None	This service does not require any authorization parameters.
Headers	Content-Type	The headers section must contain at least this header variable: Content-Type: application/x-www-form-urlencoded

Body	Form Body	`--data-urlencode 'client_id={Your machine clientId here}' \`
		`--data-urlencode 'client_secret={Your machine clientSecret here}' \`
		`--data-urlencode 'grant_type=client_credentials'`

Limitations	None	The authentication API is subject to rate limiting. If you exceed the provided rate limit for a given endpoint, you will receive the 429 Too Many Requests response with the following message: Too many requests. Check the X-RateLimit-Limit, X-RateLimit-Remaining, and X-RateLimit-Reset headers. Thresholds: • Maximum Attempts: 100 / 24 hours • Throttling Rate: 100 / 24 hours

2.2.2 Request example (Postman)

```
curl --location --globoff 'https://api.kubra.io/authentication/api/v2/token' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'client_id={Your machine clientId here}' \
--data-urlencode 'client_secret={Your machine clientSecret here}' \
--data-urlencode 'grant_type=client_credentials'
```

2.2.3 Access token response specs

Spec Category	Spec Detail	Comment
Status	Standard HTTP code	An HTTP Status code. For example: 200 (Success) 401 (Unauthorized) Full list of error codes can be found below.
Headers	Content-Type	Content-Type: application/json
Body	JSON Payload	All fields and values in the response body are represented as JSON. See below for further details.

2.2.3.1 Successful response body specs

This response is generated for HTTP response code 200.

Field	Type	Comment
access_token	string	The JWT access token. The token should be used in the Authorization header using the Bearer schema in all subsequent API calls.
token_type	string	Always contains “Bearer”.
expires_in	integer	Token expiration in seconds. Default is 86400 (24 hr).

timestamp DEPRECATED	string	Date and time at which the token was originally retrieved.
servedFromCache DEPRECATED	boolean	Boolean value indicating whether or not the token was served from the cache.

2.2.3.2 Successful response example

```
HTTP/1.1 200 OK
Content-Type: application/json
{
  "token_type": "Bearer",
  "expires_in": 86400,
  "access_token": "eyJ0eXAiOiJKV1….g1dCIVGZNNXBP",
  "timestamp": "2024-11-11T12:27:29.441Z",
  "servedFromCache": false
}
```

2.2.3.3 Failure response body specs

Field	Type	Comment
error	string	Contains error type (e.g. access_denied).
token_type	string	Contains error description (e.g. Unauthorized).
expires_in	string	(Optional) Sometimes will contain URL to additional information about the error.

2.2.3.4 Failure response example

```
HTTP/1.1 500 ERROR
Content-Type: application/json
{
    "timestamp": "2023-01-01T20:44:11.891+00:00",
    "status": 500,
    "error": "Internal Server Error",
    "path": "/authentication/api/../token"
}
```

2.3 API requests

To construct a REST API request, combine these components:

Component	Description
The HTTP method	• **GET** Requests data from a resource. • **POST** Submits data to a resource to process. • **PUT** Replaces an existing resource. • **PATCH** Partially updates an existing resource. • **DELETE** Deletes a resource.
The URL to the API service	The URL for the API service will differ depending on which service you are trying to call. You can ask your Project Manager to provide you with the details.
The URI to the resource	The resource to query, submit data to, update, or delete. You can ask your Project Manager to provide you with the details.
Query parameters	*(Optional)* Controls which data appears in the response. Used to filter, limit the size of, and sort the data in an API response. For most REST **GET** calls, you can specify one or more optional query parameters on the request URI to filter, limit the size of, and sort the data in an API response. For filter parameters, see the individual **GET** calls.
HTTP request headers	Includes the Authorization header with the access token.
A JSON request body	Required for most **POST**, **PUT**, **PATCH** calls.

2.3.1 HTTP request headers

The commonly used HTTP request headers are:

Header	Description
Accept	Required for operations with a response body. Specifies the response format. The syntax is: `Accept: application/format` Where **format** is **json**.
Authorization	Required to make API calls. To make REST API calls, include the bearer token in the **Authorization** header with the **Bearer** authentication scheme: `Authorization: Bearer {Your Access Token here}`
Content-Type	Required for operations with a request body. Specifies the request format. The syntax is: `Content-Type: application/format` Where **format** is **json**.

2.4 API responses

KUBRA API calls return HTTP status codes. Some API calls also return JSON response bodies that include information about the resource.

2.4.1 HTTP status codes

Each REST API request returns a success or error HTTP status code.

2.4.1.1 Success

In the responses, KUBRA returns **2XX** status codes for successful requests. Please see some examples below:

HTTP status code	Description
200 OK	Request is processed successfully.
201 Created	A new resource is created successfully.

2.4.1.2 Error

In the responses for failed requests, KUBRA returns HTTP **4XX** or **5XX** status codes. Please see some examples below:

HTTP status code	Description	Cause
400 Bad Request	Request is not well-formed, syntactically incorrect, or violates schema.	The server could not understand the request. It indicates one of the following conditions: • The API cannot convert the payload data to the underlying data type. • The data is not in the expected data format. • A required field is not available. • A simple data validation error occurred.
401 Unauthorized	Authentication failed due to invalid authentication credentials or expired JWT token.	The request requires authentication and the caller did not provide valid credentials.
403 Forbidden	Authorization failed due to insufficient permissions.	The client is not authorized to access this resource although it might have valid credentials.
404 Not Found	The specified resource does not exist.	The server did not find anything that matches the request URI. Either the URI is incorrect or the resource is not available. For example, no data exists in the database at that key.
405 Method Not Allowed	The server does not implement the requested HTTP method.	The service does not support the requested HTTP method.

500 Internal Server Error	An internal server error has occurred.	A system or application error occurred. Although the client appears to provide a correct request, something unexpected occurred on the server.
503 Service Unavailable	Service Unavailable.	The server cannot handle the request for a service due to temporary maintenance.

2.5 Token caching

The Authentication API implements a cache to prevent misuse of the API causing high overage charges from our auth provider. Tokens will be served from the cache if they have more than 10 minutes to expiration. Tokens that are within 10 minutes of expiration will be evicted from the cache, and a new token will be returned. The `exp` value on the token should be used as the source of truth for when that token will expire.

2.6 Troubleshooting

If you are having issues calling the endpoints, please verify that KUBRA has been whitelisted by your security team. In order to to whitelist KUBRA services, please whitelist the kubra.io domain. We do not provide specific IP addresses because they are dynamic and ever-changing.

2.7 Request headers and query parameters

Field name	From	Required	Data Type	Validation Rules (Default if optional)
KUBRA-clientId	Header	Yes	UUID	KUBRA client ID
KUBRA-realm	Header	Yes	String	`sandbox` or `live`
KUBRA-metatag-{metatag-key}	Header	No	String	This field is optional and can be excluded. You can provide multiple keys by using distinct `{metatag-key}` for each header. The following rules apply: • `{metatag-key}` cannot exceed 24 characters. • Value cannot exceed 128 characters. • All metatags combined cannot exceed 1KB.
datafeedTypeCode	Path	Yes	String	Any valid datafeed type
spaceKrn	Path	Yes	KRN (prefixed with `krn:spc:`)	Space KRN

When uploading to the Datafeed API, files must be submitted as an HTML `POST` request using the `multipart/form-data` content type.

2.8 Request payload

```
<multipart/form-data body>
```

2.8.1 Sample POST

```
POST https://api.kubra.io/datafeed/v3/balanceFeeds/{your-space-krn}/files HTTP/1.1
kubra-clientid: {your-client-id}
kubra-realm: {your-selected-realm}
kubra-metatag-{your-metatag-key}: {your-metatag-value}
Authorization: {your-machine-token}
User-Agent: PostmanRuntime/7.28.1
Accept: /
Cache-Control: no-cache
Postman-Token: <REMOVED>
Host: api.kubra.io
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Type: multipart/form-data; boundary=---119384809525703145816456
Content-Length: 227

---119384809525703145816456

Content-Disposition: form-data; name="file"; filename="foo.json"
Content-Type: application/json
```

```
{"foo": "bar"}

---119384809525703145816456--
```

A successful file upload will return a `200` status code with a response payload containing the uploaded file name.

If the datafeed does not exist, your request will return a `404` status code.

3.0 Is there sample code that helps integrate with the Datafeed API?

Here are a few sample codes in different programming languages that can facilitate integration with the Datafeed API:

3.1 VisualBasic

```vb
Imports System
Imports System.Net
Module Program
    Sub Main(args As String())
        Using client As New WebClient
            ' Add headers
            client.Headers.Add("KUBRA-ClientID", "<CLIENT_ID_GOES_HERE>")
            client.Headers.Add("KUBRA-Realm", "<REALM_GOES_HERE>")
            client.Headers.Add("KUBRA-Metatag-<METATAG_KEY>", "<METATAG_VALUE>")
            client.Headers.Add("Content-Type", "multipart/form-data; boundary=MfnBoundry")
            client.Headers.Add("Authorization", "Bearer <TOKEN_GOES_HERE>")
            ' Create body
            Dim body As String = "--MfnBoundry" & vbCrLf &
                "Content-Disposition: form-data; name=""file""; filename=""test.jsonl""" & vbCrLf &
                "Content-Type: application/jsonl" & vbCrLf &
                "" & vbCrLf &
                "<FILE_CONTENT_GOES_HERE>" & vbCrLf &
                "--MfnBoundry--"
            ' Send data
            Dim Response = client.UploadString("https://api.kubra.io/datafeed/v3/<DATAFEED_TYPE>/<SPACE_KRN>/files", body)
            Console.Out.WriteLine(Response)
        End Using
    End Sub
End Module
```

3.2 C#

```cs
using System.Net;
namespace FileUpload
{
    internal class Program
    {
        static void Main(string[] args)
        {
            WebClient client = new WebClient();
            // add headers
            client.Headers.Add("KUBRA-ClientID", "<CLIENT_ID_HERE>");
            client.Headers.Add("KUBRA-Realm", "<REALM_HERE>");
            client.Headers.Add("KUBRA-Metatag-<METATAG_KEY_HERE>", "<METATAG_VALUE_HERE>");
            client.Headers.Add("Content-Type", "multipart/form-data; boundary=MfnBoundry");
            client.Headers.Add("Authorization", "Bearer <TOKEN_HERE>");
            // create body
            String body = "--MfnBoundry\r\n" +
               "Content-Disposition: form-data; name=\"file\"; filename=\"test.jsonl\"\r\n" +
               "Content-Type: application/jsonl\r\n" +
               "\r\n" +
               "<FILE_CONTENT_HERE>\r\n" +
               "--MfnBoundry--";
            // send data
            string Response = client.UploadString("https://api.kubra.io/datafeed/v3/<DATAFEED_TYPE>/<SPACE_KRN>/files", body);
            Console.WriteLine(Response);
        }
    }
}
```

3.3 Java

```java
import java.io.OutputStream;
import java.io.OutputStreamWriter;
import java.io.PrintWriter;
import java.net.HttpURLConnection;
import java.net.URLConnection;
import java.net.URL;
```

```java
public class FileUpload {
    public static void main(String[] args) {
        String url = "https://api.kubra.io/datafeed/v3/<DATAFEED_TYPE>/<SPACE_KRN>/files";
        String charset = "UTF-8";
        String crlf = "\r\n";
        try  {
            URLConnection connection = new URL(url).openConnection();
            connection.setDoOutput(true);
            // set headers
            connection.setRequestProperty("KUBRA-ClientID", "<CLIENT_ID>");
            connection.setRequestProperty("KUBRA-Realm", "<REALM>");
            connection.setRequestProperty("KUBRA-Metatag-<METATAG-KEY>", "<METATAG-VALUE>");
            connection.setRequestProperty("Content-Type", "multipart/form-data; boundary=MfnBoundry");
            connection.setRequestProperty("Authorization", "Bearer <TOKEN>");
            OutputStream output = connection.getOutputStream();
            PrintWriter writer = new PrintWriter(new OutputStreamWriter(output, charset), true);
            // send body with file data
            writer.append("--MfnBoundry").append(crlf);
            writer.append("Content-Disposition: form-data; name=\"file\"; filename=\"test.jsonl\"").append(crlf);
            writer.append("Content-Type: application/jsonl").append(crlf);
            writer.append(crlf);
            writer.append("<FILE_CONTENT>");
            writer.append(crlf);
            writer.append( "--MfnBoundry--").flush();
            // get response
            int responseCode = ((HttpURLConnection) connection).getResponseCode();
            System.out.println(responseCode);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3.4 VS Code REST Plugin

```
@SERVER=https://api.kubra.io
@AUTH0=Bearer <KEY_HERE>
@TENANT=<CLIENT_ID_HERE>
@REALM=<REALM_HERE>
@FEED_TYPE=<FEED_TYPE_HERE>
@SPACE_KRN=<SPACE_KRN_HERE>
@METATAG_KEY=<METATAG_KEY_HERE>
@METATAG_VALUE=<METATAG_VALUE_HERE>
###
POST {{SERVER}}/datafeed/v3/{{FEED_TYPE}}/{{SPACE_KRN}}/files HTTP/1.1
Authorization: {{AUTH0}}
KUBRA-ClientID: {{TENANT}}
KUBRA-Realm: {{REALM}}
KUBRA-Metatag-{{METATAG_KEY}}: {{METATAG_VALUE}}
Content-Type: multipart/form-data; boundary=MfnBoundry
--MfnBoundry
Content-Disposition: form-data; name="file"; filename="test.jsonl"
Content-Type: application/jsonl
< <FILE_NAME_GOES_HERE>
--MfnBoundry--
```

4.0 How do KUBRA applications process uploaded files?

Downstream services, such as accounts and balances, will automatically process files uploaded to their associated datafeeds.

5.0 Troubleshooting

For troubleshooting, talk to your KUBRA Project Manager or Service Delivery Team member.